03 · Participants & models

Siddharth M. Bhatia · siddharth@sidmb.com
July 1, 2026

Contents

How models resolve 1
Load weights directly 2
Models too big for one GPU 3
`ModelParticipant` knobs 4
`kv_reuse` (cross-turn KV cache) 4
API-backed participants 5
Batch mode for large rollouts 6
Per-turn `max_new_tokens` 7

How models resolve

There are **no short names** — you pass an **HF id or local path** (`str | Path`, or an already-loaded model), and interlens picks the family-correct participant class from the model's own **`config.model_type`** (exactly like HuggingFace `AutoModel`), then derives its chat-template flags by probing the tokenizer. Any model of a supported family Just Works:

```
from interlens import AutoModelParticipant
p = AutoModelParticipant.from_pretrained("google/gemma-2-2b-it", name="p")
     # → GemmaModelParticipant
q = AutoModelParticipant.from_pretrained("Qwen/Qwen2.5-3B-Instruct",
     name="q")  # → QwenModelParticipant
r = AutoModelParticipant.from_pretrained("mistralai/Mistral-7B-Instruct-
     v0.3", name="r")  # → base ModelParticipant
```

The `config.model_type` → class mapping is a **class self-registry**: each participant subclass declares the model types it handles in a `MODEL_TYPES` class attribute, and `ModelParticipant.__init_subclass__` records them — there is no central table to keep in sync. Unregistered families (Mistral, OLMo, Phi, DeepSeek, …) fall back to the base `ModelParticipant` automatically, and a brand-new family/size resolves the moment it's on the Hub.

Weights load lazily. `from_pretrained` does **not** touch weights — it reads only the model's `config.json` to resolve the family, and defers the actual load to the first time the model is used (or `p.model`/`p.tokenizer` is accessed). So an unbuilt participant is a cheap recipe: it costs KBs, pickles across a spawn boundary without shipping weights (each GPU worker loads on its own device), and `.set(...)` copies share one loaded model object by reference. The load is process-cached on `(hf_id, device, dtype, attn, quant, revision)`, so every same-recipe participant on a device shares ONE object — which is exactly what lets the runner batch them into a single `model.generate`. The default device is `cuda`; the runner binds each participant's device per worker before the load fires.

Static types when you want them. `AutoModelParticipant.from_pretrained` is *dynamically* dispatched, so its declared return type is the base `ModelParticipant` — except for known id literals, which a type stub (`factories.pyi`) narrows to the concrete subclass. For a guaranteed static type regardless of the id, **name the class directly** — it returns `Self`:

```
from interlens import QwenModelParticipant
q = QwenModelParticipant.from_pretrained("Qwen/Qwen2.5-3B-Instruct",
     name="q")  # statically QwenModelParticipant
```

Family behavior is data-driven, not hand-declared. Whether a template accepts a standalone system role or requires strictly alternating turns is **auto-derived from the tokenizer** (`interlens.loading.derive_chat_flags`), so e.g. Gemma 2 (folds system into the first user turn) and Gemma 3 (accepts a system role) are handled correctly with zero per-model config. A subclass exists only when a family needs distinct behavior — Gemma's `tool_code` and Llama's `◁python_tag▷` tool-call formats — or, for Qwen, purely so its models get a distinct statically-typed class (its `<tool_call>` JSON is already the base format). A slow test (`tests/test_family_flags.py`) checks the derivation against real tokenizers.

Load weights directly

```
import torch
from interlens.loading import load_model
```

```
model, tok = load_model("Qwen/Qwen3-8B", device="cuda",
     dtype=torch.bfloat16, attn="flash_attention_2")
```

`load_model` shares a process-local cache: identical `(hf_id, device, dtype, attn, quant, revision, max_memory)` returns the same model object; each HF id caches its own tokenizer. Flash-attention is the default with automatic fallback to sdpa/eager; `quant="4bit"`/`"8bit"` is opt-in (perturbs activations → interp fidelity).

Models too big for one GPU

Pass a **sharding strategy** as `device` and it is routed to `device_map=` (the `.to(device)` is skipped, since moving a sharded model would undo the placement). `max_memory` reserves per-device headroom — worth setting whenever you shard, because the automatic map sizes for *weights* while your peak is usually KV cache and activations.

```
model, tok = load_model(
    "Qwen/Qwen3-32B",
    device="balanced",                                    # or
     "auto" / "balanced_low_0" / "sequential" / a {module: device} dict
    max_memory={0: "70GiB", 1: "70GiB", "cpu": "0GiB"},        # "cpu":
     "0GiB" forbids offload → an actionable OOM instead of a silent 100x
     slowdown
)
```

Once a model is sharded, `model.device` is **not** where inputs go — the input embedding's device is. Use `input_device`; it returns the single device unchanged for an ordinary load, so the same code covers both:

```
from interlens.loading import input_device, is_sharded
ids = tok("hello", return_tensors="pt").input_ids.to(input_device(model))
is_sharded(model)      # True only when device_map spread it over more
     than one device
```

`ModelParticipant` does this for you (`p.input_device`), as do `capture_router_logits` and `span_pooled_residuals`.

`ModelParticipant` knobs

```
from interlens import AutoModelParticipant
p = AutoModelParticipant.from_pretrained(
    "Qwen/Qwen3-4B", name="p", device="cuda",
    load_kwargs={"attn": "sdpa"},   # optional: forwarded to load_model
     (dtype/attn/quant/revision)
    temperature=0.8, top_p=0.95, max_new_tokens=512,
    seed=1234,                # per-participant RNG seed → reproducible
     greedy/sampled turns (local models only)
    thinking="auto",          # "auto" defers to the template; True/False
     forces enable_thinking where supported
    system_prompt="…",
    kv_reuse="auto",          # cross-turn KV prefix reuse; see below
)
```

Already hold weights (e.g. sharing them, or an externally-loaded checkpoint)? Wrap them with `from_model`:

```
from interlens.loading import load_model
model, tok = load_model("Qwen/Qwen3-4B")
p = AutoModelParticipant.from_model(model, tok, name="p", temperature=0.8)
     # family from model.config.model_type
```

`kv_reuse` (cross-turn KV cache)

`"auto"` (default) reuses the KV cache across a speaker's own consecutive turns when the new prompt exactly extends the cached tokens — skipping a full re-prefill. It is doubly guarded (exact-prefix check + safe fallback) and **auto-disables under steering/patch and batched generation**.

```
p.kv_reuse = "auto"   # default: enabled when safe
p.kv_reuse = False    # force off — pin this for determinism-critical /
     reproducibility experiments,
                      #   since reuse can perturb outputs at the FP level
     vs a full prefill.
```

Enable `logging` at INFO to see the per-participant decision, DEBUG to see reuse engage per turn:

```
import logging; logging.basicConfig(level=logging.INFO)
```

```
# INFO  ...model_participant: p: cross-turn KV reuse ENABLED
    (kv_reuse='auto')
```

API-backed participants

`APIParticipant` is a full conversational participant with **no local model** — use it as an opponent, moderator, or judge. Interp requests (`capture`/`steering`/`patch`/`return_logprobs`) **raise** rather than silently no-op (a steering sweep that quietly did nothing would fabricate a "no effect" result).

```
from interlens import Conversation, APIParticipant, AutoModelParticipant
local = AutoModelParticipant.from_pretrained("Qwen/Qwen2.5-3B-Instruct",
    name="student")
judge = APIParticipant(name="tutor", model_id="claude-sonnet-5",
    provider="anthropic",
                       system_prompt="You are a patient tutor. Ask one
    probing question per turn.",
                       max_tokens=400, temperature=1.0)
conv = Conversation(participants=(local, judge), shared_context="Teach the
    student about entropy.")
conv.run(turns=6)
```

Requires `ANTHROPIC_API_KEY` (or `ANTHROPIC_API_KEY_FILE`) and outbound network. For tests, inject a fake `client=callable(system, messages, model, max_tokens, temperature) → str`.

OpenAI (`provider="openai"`) calls OpenAI directly via the `openai` SDK — needs `OPENAI_API_KEY` and the `[api]` extra:

```
judge = APIParticipant(name="judge", provider="openai", model_id="gpt-5",
    max_tokens=400)
```

OpenRouter (`provider="openrouter"`) reaches any model behind openrouter.ai through one OpenAI-compatible endpoint — needs `OPENROUTER_API_KEY` and the `[api]` extra (`pip install "interlens[api]"`, which pulls `openai`):

```
from interlens import OpenRouterRouting
```

```
judge = APIParticipant(name="judge", provider="openrouter",
                       model_id="meta-llama/llama-3.1-70b-instruct",   #
     or openai/gpt-5, anthropic/claude-sonnet-5, …
                       openrouter_routing=OpenRouterRouting(
                           upstream_provider="together",
                           quantizations=("bf16",),
                           data_collection="deny",
                       ),
                       max_tokens=400)
```

OpenRouter routing is fail-closed for research: a request must pin one upstream provider (fallbacks are disabled and parameter support is required), or explicitly opt into uncontrolled exploratory routing with `OpenRouterRouting.unpinned()`. Each returned turn records the requested routing object, actual upstream provider, resolved model, and generation ID in `Message.metadata`; a missing or mismatched provider aborts the turn.

All providers share one retry/backoff + max-in-flight client, built lazily per provider.

Batch mode for large rollouts

`provider="anthropic"` and `provider="openai"` expose asynchronous **batch APIs** (Anthropic Message Batches / OpenAI Batch) — ~50% cost and much higher throughput, in exchange for batch-window latency. Set `batch=True` on the participant and run the rollout in throughput mode; the co-stepper collects each round's same-position turns across all rollouts into **one** provider batch:

```
pro = APIParticipant(name="pro", provider="openai", model_id="gpt-5",
     batch=True, system_prompt="Argue YES.")
con = APIParticipant(name="con", provider="anthropic", model_id="claude-
     sonnet-5", batch=True, system_prompt="Argue NO.")
report = (Conversation(participants=[pro, con], shared_context="…")
          .turns(4).rollout(n=200))   # batched=True is the default; each
     round → one batch per speaker
```

Batch mode is **unavailable on OpenRouter** — requesting `batch=True` there raises rather than silently degrading to serial calls, so a requested batch discount is never quietly dropped. Under the hood every participant/client also exposes `generate_batch(views)` / `client.submit_batch(requests)` directly if you want to batch outside a rollout.

Per-turn `max_new_tokens`

`temperature`/`top_p`/`seed` are set once per participant, but generation *length* often varies per turn (a one-word verdict vs a paragraph). `step`/`sample` take an optional `max_new_tokens` override for exactly that:

```
conv.sample("a", "Answer in one word.", max_new_tokens=8)     # short
conv.step(conv.participant("a"), max_new_tokens=400)             # long
```

Next: context management & serialization.

© 2026 Siddharth M. Bhatia. All rights reserved. Interlens is licensed under the GNU Affero General Public License v3.0.